SOLUTION

Experiment 1: Decision Tree Classification

Aim

To implement and evaluate a Decision Tree Classifier using the Iris dataset and analyze its classification performance.

Procedure

1. Load the Iris dataset using Scikit-learn.
2. Check the first five rows and data types.
3. Check and handle missing values.
4. Separate the features and target variable.
5. Split the dataset into training and testing sets.
6. Build a Decision Tree using the **Gini Index** as the splitting criterion.
7. Train the classifier using the training data.
8. Display the decision tree structure and identify the root node.
9. Predict the classes of the testing data.
10. Evaluate the model using a confusion matrix and classification report.

Python Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

import matplotlib.pyplot as plt

import pandas as pd


# Load dataset

iris = load_iris()
```

```
# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Features and target
X = df[iris.feature_names]
y = df["target"]

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create Decision Tree
model = DecisionTreeClassifier(
```

```
    criterion="gini",
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

```
# Display root node
root_feature = iris.feature_names[model.tree_.feature[0]]
print("\nRoot Node Feature:", root_feature)

# Display tree
plt.figure(figsize=(15, 8))
plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
plt.title("Decision Tree Classification")
plt.show()

# Display misclassified instances
misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

print("\nMisclassified Instances:")
print(misclassified)
```

Expected Result

The Decision Tree successfully classifies the Iris flower

Experiment 2: Q-Learning – 4×4 Grid World

Aim

To implement and evaluate a **Q-Learning reinforcement learning algorithm** using a 4×4 Grid World and analyze the agent's learning performance.

Procedure

1. Create a **4×4 Grid World** containing 16 states.
2. Define the starting state, goal state, and obstacle state.
3. Define four possible actions: **Up, Down, Left, and Right**.
4. Assign suitable rewards and penalties for different actions.
5. Initialize the Q-table with zeros.
6. Set the learning rate, discount factor, and exploration rate.
7. Use the **ε-greedy strategy** to select actions.
8. Update the Q-table using the Q-Learning update equation.
9. Train the agent for at least **100 episodes**.
10. Record the Q-values at Episodes **1, 50, and 100**.
11. Display the final Q-table and learned policy.
12. Plot the cumulative reward for each episode.
13. Analyze the learning and convergence of the agent.

Python Code

```
import numpy as np
import random
import matplotlib.pyplot as plt


# ----------------------------
# Q-Learning - 4x4 Grid World
# ----------------------------


GRID_SIZE = 4
```

```
NUM_STATES = 16

# Grid configuration
START_STATE = 0
GOAL_STATE = 15
OBSTACLE_STATE = 5

# Actions
UP = 0
DOWN = 1
LEFT = 2
RIGHT = 3

actions = ["UP", "DOWN", "LEFT", "RIGHT"]

# Q-Learning parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 100
max_steps = 100

# Initialize Q-table
Q = np.zeros((NUM_STATES, 4))

# Store Q-values at selected episodes
```

```
q_episode_1 = None
q_episode_50 = None
q_episode_100 = None

# Store rewards
episode_rewards = []



# ----------------------------
# Find Next State
# ----------------------------
def get_next_state(state, action):

    row = state // GRID_SIZE
    col = state % GRID_SIZE

    new_row = row
    new_col = col

    if action == UP:
        new_row -= 1

    elif action == DOWN:
        new_row += 1

    elif action == LEFT:
        new_col -= 1
```

```
    elif action == RIGHT:
        new_col += 1

    # Prevent movement outside the grid
    if new_row < 0 or new_row >= GRID_SIZE:
        return state

    if new_col < 0 or new_col >= GRID_SIZE:
        return state

    return new_row * GRID_SIZE + new_col



# ----------------------------
# Reward Function
# ----------------------------
def get_reward(next_state):

    if next_state == GOAL_STATE:
        return 10

    elif next_state == OBSTACLE_STATE:
        return -5

    else:
        return -1
```

```
# ----------------------------
# Q-Learning Training
# ----------------------------
for episode in range(1, episodes + 1):

    state = START_STATE
    total_reward = 0

    for step in range(max_steps):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])

        # Find next state
        next_state = get_next_state(state, action)

        # Get reward
        reward = get_reward(next_state)

        # Q-Learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward
```

```
            + gamma * np.max(Q[next_state])
            - Q[state, action]
        )

        total_reward += reward

        # Move to next state
        state = next_state

        # Stop when goal is reached
        if state == GOAL_STATE:
            break

    episode_rewards.append(total_reward)

    # Store Q-values
    if episode == 1:
        q_episode_1 = Q.copy()

    elif episode == 50:
        q_episode_50 = Q.copy()

    elif episode == 100:
        q_episode_100 = Q.copy()


# ----------------------------
```

```
# Display Q-Values
# ----------------------------
np.set_printoptions(precision=2, suppress=True)

print("=" * 50)
print("Q-VALUES AT EPISODE 1")
print("=" * 50)
print(q_episode_1)

print("\n" + "=" * 50)
print("Q-VALUES AT EPISODE 50")
print("=" * 50)
print(q_episode_50)

print("\n" + "=" * 50)
print("Q-VALUES AT EPISODE 100")
print("=" * 50)
print(q_episode_100)



# ----------------------------
# Final Q-Table
# ----------------------------
print("\n" + "=" * 50)
print("FINAL Q-TABLE")
print("=" * 50)
print(Q)
```

```
# ----------------------------
# Display Learned Policy
# ----------------------------
print("\n" + "=" * 50)
print("FINAL LEARNED POLICY")
print("=" * 50)

symbols = {
    UP: "↑",
    DOWN: "↓",
    LEFT: "←",
    RIGHT: "→"
}

for row in range(GRID_SIZE):

    policy_row = []

    for col in range(GRID_SIZE):

        state = row * GRID_SIZE + col

        if state == GOAL_STATE:
            policy_row.append("G")
```

```
        elif state == OBSTACLE_STATE:
            policy_row.append("X")

        else:
            best_action = np.argmax(Q[state])
            policy_row.append(symbols[best_action])

    print(" | ".join(policy_row))



# ----------------------------
# Display Grid World
# ----------------------------
print("\n" + "=" * 50)
print("GRID WORLD")
print("=" * 50)

for row in range(GRID_SIZE):

    grid_row = []

    for col in range(GRID_SIZE):

        state = row * GRID_SIZE + col

        if state == START_STATE:
            grid_row.append("S")
```

```
        elif state == GOAL_STATE:
            grid_row.append("G")

        elif state == OBSTACLE_STATE:
            grid_row.append("X")

        else:
            grid_row.append(str(state))

    print(" | ".join(grid_row))



# ----------------------------
# Plot Cumulative Reward
# ----------------------------
plt.figure(figsize=(10, 5))

plt.plot(
    range(1, episodes + 1),
    episode_rewards,
    marker="o",
    markersize=3
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
```

```
plt.title("Q-Learning - Cumulative Reward per Episode")
plt.grid(True)


plt.tight_layout()
plt.show()
```

Expected Result

The Q-Learning agent successfully learns a suitable path from the **starting state (0)** to the **goal state (15)** while avoiding the obstacle at **state 5**.

The program displays:

- Q-values at **Episode 1**
- Q-values at **Episode 50**
- Q-values at **Episode 100**
- Final Q-table
- Final learned policy
- Grid World representation
- Cumulative reward graph